

BANK MANAGEMENT SYSTEM

PROJECT REPORT

Prepared by: Bhavya Singh

1. Introduction

The Bank Management System is a Python-based graphical user interface (GUI) application developed to demonstrate basic banking operations in a simple and user-friendly environment. The application provides separate functionality for users and administrators and demonstrates the practical use of Python programming concepts such as functions, conditional statements, dictionaries, lists, validation, GUI programming, and data visualization.

2. Problem Statement

Managing basic banking activities through separate manual processes can be time-consuming and difficult to organize. This project provides a simple computerized system for performing common banking operations such as viewing account information, depositing and withdrawing money, checking loan information, maintaining a passbook, and managing account-related data.

3. Objectives

- To develop a simple Bank Management System using Python.
- To create a user-friendly graphical interface using Tkinter.
- To provide basic deposit and withdrawal operations.
- To allow users to view and update account information.
- To provide administrators with account and loan management functions.
- To display loan information through graphical visualization.
- To demonstrate input validation and error handling.
- To store project data using Python data structures without MySQL.

4. Technologies Used

- Python – Main programming language.
- Tkinter – Used to create the graphical user interface.
- ttk – Used for themed GUI widgets.
- Matplotlib – Used to create the loan graph.
- Datetime – Used for date and time related information.
- Random – Used for random number generation.

5. System Features

5.1 User Features

- User login.
- View account details.
- Update name, email, phone number, and address.
- Deposit money.
- Withdraw money.
- View passbook/transaction information.
- Check loan status.

- View loan graph.
- Submit feedback.

5.2 Admin Features

- Administrator login.
- Add account information.
- View loan data.
- Update loan status.
- View loan defaulters.
- View user feedback.
- Add loan data.
- View loan graph.
- Perform deposit and withdrawal operations.

6. System Design

The application starts with a main screen and provides login-based access to the banking functions. After authentication, users are directed to the user dashboard while the administrator is directed to the admin dashboard. Each dashboard contains buttons and controls for the available operations.

7. Data Storage

The project uses Python dictionaries and lists to store account, user, loan, and related information. No external database such as MySQL is used. The application is therefore a Python-only project and is intended primarily for educational demonstration.

8. Loan Graph and Data Visualization

The project uses Matplotlib to display a loan graph. The graph represents loan-related data over the years 2017 to 2026. This demonstrates how Python can be used not only for data processing but also for visual representation of information.

9. Input Validation and Error Handling

- Checks are performed on important user inputs.
- Phone number input is validated.
- Transaction amounts are checked before processing.
- Account existence is checked for relevant operations.
- Balance-related conditions are considered during transactions.
- Message boxes are used to communicate errors and results to the user.

10. Login Details for Demonstration

The source code contains pre-stored demonstration login credentials.

Role/User	Username	Password
-----------	----------	----------

Admin	admin	admin123
Rahul Sharma	rahul	1234
Priya Singh	priya	5678
Aman Verma	aman	1111

11. Project Workflow

- Start the Python program.
- The main screen is displayed.
- Select the appropriate login option.
- Enter the required credentials.
- Access the corresponding user or admin dashboard.
- Select a banking operation.
- Enter or update the required information.
- The system validates the input and displays the result.
- The user can return to the dashboard or exit the application.

12. Testing

The application can be tested by running the Python file and checking the major functions available from the user and admin dashboards. Testing includes login, account viewing and updating, deposit and withdrawal operations, loan-related functions, feedback, passbook information, and graph display.

- Login functionality tested with available demonstration credentials.
- Account information display checked.
- Account information update functions checked.
- Deposit and withdrawal operations checked.
- Loan status and loan graph functions checked.
- Admin loan and feedback functions checked.
- Input validation and message-box responses checked.

13. Advantages

- Simple and easy-to-use GUI.
- Demonstrates multiple Python concepts in one project.
- Provides separate user and administrator functions.
- Includes basic validation.
- Includes graphical data visualization.
- Does not require an external database for the demonstration.

14. Limitations

- Data is stored in Python data structures rather than a permanent database.
- The login credentials are pre-stored in the source code.
- The system is designed for educational use rather than real banking.
- It does not provide the security infrastructure required by real banking systems.

- The project does not use a live banking or financial service.

15. Future Scope

- Integrate a database such as MySQL or SQLite.
- Add stronger authentication and password security.
- Add permanent transaction history.
- Provide more detailed reports and analytics.
- Improve account and loan management features.
- Add role-based access control and additional security measures.

16. Conclusion

The Bank Management System successfully demonstrates how Python can be used to create a graphical application for basic banking operations. The project combines Tkinter for the GUI, Python data structures for storing information, validation for user input, and Matplotlib for data visualization. It provides separate user and administrator functions and serves as an educational example of applying Python programming concepts to a practical application.

17. Project File

Main source file: BANK MANAGEMENT SYSTEM.py

Supporting project documentation: README.md and statement.md

18. Declaration

This project has been developed for educational purposes to demonstrate programming concepts and the development of a basic graphical banking application. It should not be considered a production-ready banking system.